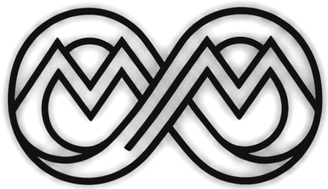


MLOPS ENGINEERING · SESSION 1 OF 5

From Code to Container

Packaging · FastAPI · Docker · pytest · Logging

Aya · MLOps MENA Community · · ~3 hours

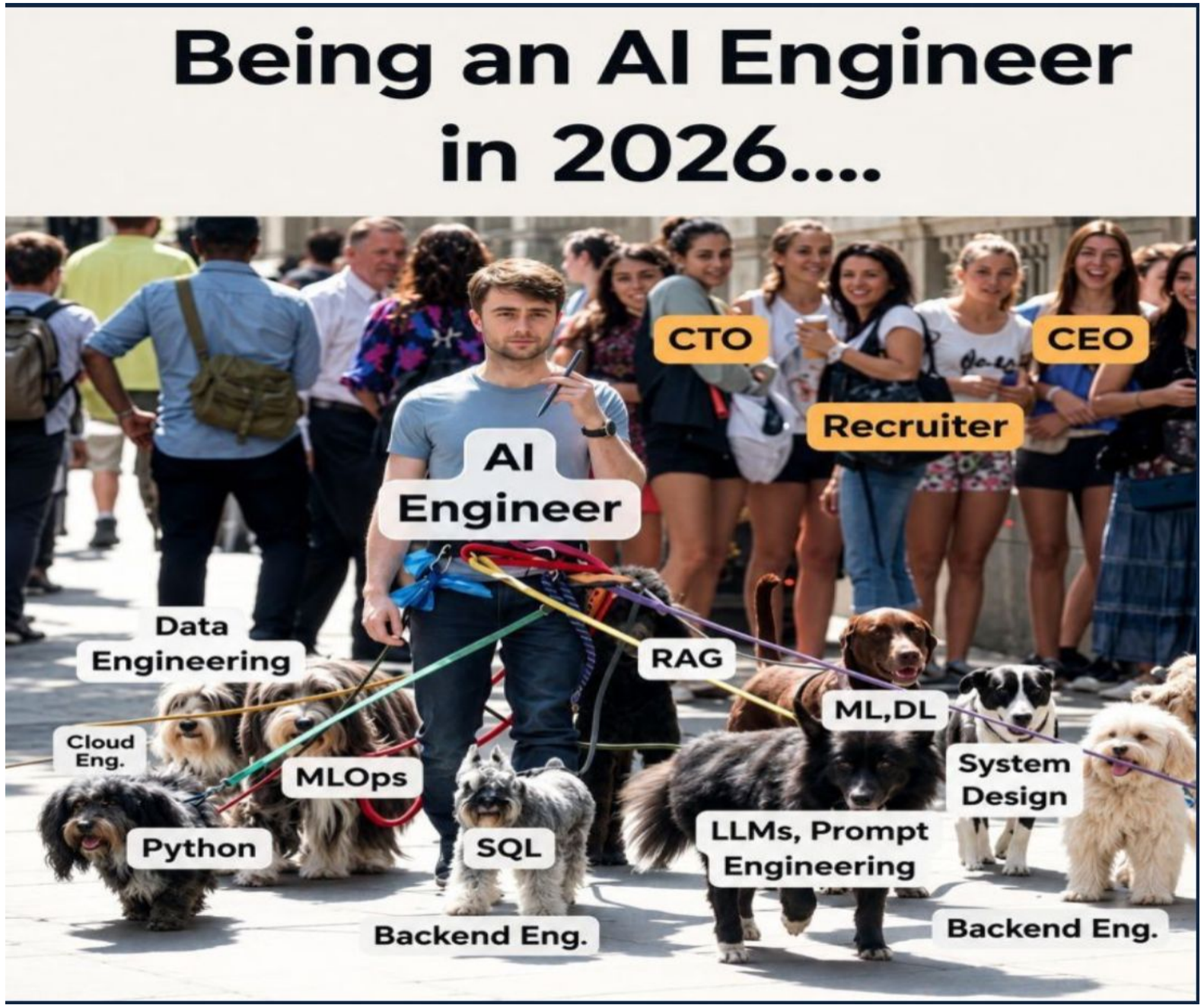
WHO I AM

Aya Nasser Salama

Founder of MLOPs MENA Community, Senior MLOps Engineer

A mother, an Engineer, a master student, a volunteer

إن العلم أمانة، وإن ما نتعلمه هنا لا نُبقيه لأنفسنا بل ننقله لغيرنا



This course gives you the leashes.

WHY MLOPS

Seven reasons this is worth your evening

01

You are already doing it — badly

02

The job market now requires it

03

Highest-paid engineering specialization

04

Models are easy. Deployment is the hard part

05

Companies need MLOps to cut AI costs

06

DevOps engineer? You already have 60%

28% salary premium for MLOps skills Burtch Works 2024

07

Be the joker card your team cannot afford to lose

87% of ML models never reach production Gartner 2024

The question is not whether to learn MLOps. It is whether you learn it now.

SESSION 1 · AGENDA

What we cover today

Eight topics · 2.5–3 hours · ends with a project

01	**MLOps Maturity Model**	Why MLOps exists and where teams struggle
02	**Python Packaging & Structure**	pyproject.toml, OOP, type hints, decorators
03	**FastAPI**	REST APIs for ML inference
04	**Serialization Formats**	JSON, Protobuf, Pickle, ONNX
05	**Docker & Containerization**	Dockerfiles, multi-stage builds, docker-compose
06	**Structured Logging**	JSON logs, correlation IDs, log levels
07	**Unit Testing with pytest**	Fixtures, mocks, coverage
08	**Session Project**	Containerized ML API with a full test suite

ABOUT

About this course

5

Sessions

Phase 1 → 4 of the MLOps roadmap

12-15h

Total hours

2.5–3 hours per session

100%

Hands-on

Every topic ends in working code

Free

Always

All tools are open-source

Content aligned with the MLOps Zoomcamp by DataTalks.Club · github.com/DataTalksClub/mlops-zoomcamp

TOPIC 01

MLOps Maturity Model

Where your team sits today, and the next concrete step

Where does your team sit today?

A diagnostic tool for the next concrete step — not a destination to reach overnight.

MLOps

Productionize models reliably, reproducibly, and at scale

DevOps

Ship software reliably — build, test, deploy code

DataOps

Reliable, fast data pipelines from source to consumer

DIAGNOSE YOUR TEAM

Can you reproduce any past experiment from the artifact store alone?

How long does a model go from trained to serving predictions?

Who gets paged when the model silently starts performing poorly?

The five levels

Level 0

No MLOps

Notebooks only. Manual everything. No reproducibility.

Level 1

Manual Process

Scripts exist but deployments are manual. No tracking.

Level 2

ML Pipeline

Automated training. Experiment tracking. Basic CI.

Level 3

CD for ML

Auto retraining on trigger. Model registry. Monitoring.

Level 4

Full MLOps

Zero-touch pipelines. Auto rollback. Full observability.

Most teams start at Level 0 or 1. This session takes you to Level 2.

“Your model is done. Now what?”

The model works in the notebook. Can anyone else use it?

notebook_final_v3_FINAL_USE_THIS.ipynb

```
import pandas as pd
import pickle

df = pd.read_csv('/Users/ahmed/Desktop/data_final.csv')

# trained last week, don't retrain
model = pickle.load(open('model_v2_good.pkl','rb'))

# this works on my machine
pred = model.predict(df[features])
```

Hardcoded path

Breaks on any other machine.

No input validation

Crashes on unexpected data.

No API

Nobody else can call it.

No version

Which model is in production right now?

TOPIC 01 · THIS SESSION

By the end of today you will have

1

A proper Python package

src/ layout, installable

2

A validated REST API

/predict and /health

3

A Docker container

Same on every machine

4

A passing test suite

pytest, ≥ 80% coverage

5

Structured logs

JSON, searchable

Why → what → how → result. The Docker image you build today is the input to Session 2.

TOPIC 02

Packaging and project structure

Stop working in notebooks alone

Python project structure

Six directories, and one rule for each.

my_ml_project/

```
src/
  __init__.py
  model.py
  pipeline.py
  utils.py
tests/
  test_model.py
configs/
  config.yaml
notebooks/
  exploration.ipynb
Dockerfile
pyproject.toml
README.md
```

`src/`	Installable package — import your own code cleanly
`tests/`	Auto-discovered by pytest — CI-ready from day 1
`configs/`	Separate config from code — no magic constants
`notebooks/`	Exploration only — never import from notebooks
`Dockerfile`	Reproducible runtime — same env everywhere
`pyproject.toml`	Single source of truth for deps and tools

pyproject.toml

One file to rule all your tooling

pyproject.toml

```toml
[project]
name = "my-ml-project"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = ["scikit-learn", "fastapi", "pydantic"]

[project.optional-dependencies]
dev = ["pytest", "ruff", "black"]

[tool.ruff]
line-length = 88

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-v --cov=src"
```

Project metadata

Name, version, Python constraint.

Dev deps separated

Production images stay small.

Ruff config

Replaces Flake8 and isort.

Install editable

pip install -e .[dev]

OOP and type hints

Write code your future self can debug

src/model.py

```python
from abc import ABC, abstractmethod

class ModelBase(ABC):
    @abstractmethod
    def predict(self, X: list[float]) -> float: ...

class RideDurationModel(ModelBase):
    """Predicts NYC taxi ride duration in minutes."""

    def __init__(self, threshold: float = 60.0) -> None:
        self.threshold = threshold
        self._model = None

    def predict(self, X: list[float]) -> float:
        pred = self._model.predict([X])[0]
        return min(pred, self.threshold)
```

Abstract base class

Forces every model to implement .predict().

Type hints

list[float] → float. IDEs catch bugs before runtime.

Docstrings

Document intent, not implementation.

Composition

Keep _model private, not a parent class.

TOPIC 03

From notebook to REST API

Because a function is not an endpoint

Why a framework?

WITHOUT A FRAMEWORK

- No input validation — bad data reaches your model
- No HTTP handling — you write sockets from scratch
- No docs — nobody knows how to call your endpoint
- No async — one user blocks all others

FASTAPI GIVES YOU

- Pydantic schemas — invalid requests rejected at the door
- HTTP and HTTPS handling built in
- Auto-generated /docs — interactive UI instantly
- Async support — concurrent requests handled properly

Your model as a REST API

About twenty lines

src/api.py

```python
from fastapi import FastAPI
from pydantic import BaseModel, Field
from src.model import RideDurationModel

app = FastAPI(title="Ride Duration API")
model = RideDurationModel()   # loaded once

class PredictRequest(BaseModel):
    distance_km: float = Field(..., gt=0)
    passengers: int = Field(1, ge=1, le=8)

class PredictResponse(BaseModel):
    duration_min: float

@app.post("/predict", response_model=PredictResponse)
async def predict(req: PredictRequest):
    d = model.predict([req.distance_km, req.passengers])
    return PredictResponse(duration_min=round(d, 2))
```

Pydantic validation

Field(gt=0) rejects negative distances. 422 before predict() runs.

Load the model once

At module level. Inside predict() it reloads on every request.

response_model

FastAPI validates your response too — it fails loudly in dev.

Run it

uvicorn src.api:app --reload → localhost:8000/docs

What every ML endpoint must have

01	**Input validation**	Reject malformed data before it reaches the model	`Pydantic BaseModel with Field constraints`
02	**Health endpoint**	Load balancers need /health to route traffic	`@app.get("/health") returning {status: healthy}`
03	**Model loaded once**	Loading on every request adds 200–500ms	`Instantiate at module level, reuse across requests`
04	**Async handlers**	Sync handlers block the server under load	`async def predict()`
05	**Response schema**	Unvalidated responses hide bugs until production	`response_model=PredictResponse`
06	**Auto-generated docs**	Your API is useless if nobody can call it	`/docs and /redoc, zero code`

FastAPI and Litestar both give you all six. We compare them under load in Session 3.

TOPIC 04

Serialization and data validation

How your data travels between services

Serialization formats

FORMAT	TYPE	SPEED	SIZE	HUMAN READABLE	BEST FOR
JSON	Text	Slow	Large	Yes	REST APIs, configs, logging
MessagePack	Binary	Fast	Small	No	High-throughput inter-service calls
Protobuf	Binary	Fast	Tiny	No	gRPC services, streaming pipelines
Pickle	Binary	Fast	Medium	No	Python-only model files — never in APIs
ONNX	Binary	—	Medium	No	Cross-framework model portability

Never serialize models with Pickle and expose them via an API. Pickle is not safe to deserialize from untrusted sources.

What is ONNX?

A common computation-graph format, so a model trained in one framework runs in a completely different runtime.

1. Train
PyTorch, TF, sklearn, XGBoost

2. Export
Convert to the .onnx graph

3. Optimize
Quantize, fuse ops, prune

4. Run anywhere
C++, Java, JS, mobile, edge, GPU

Framework independence

Train in PyTorch, serve with ONNX Runtime — no PyTorch in production.

Smaller production image

Drop training frameworks. Ship onnxruntime only, about 15MB.

Cross-language serving

Run the same file from C++, Java, C#, JavaScript or Rust.

Hardware acceleration

Execution providers target CUDA, TensorRT, OpenVINO, CoreML.

Exporting a PyTorch model

From trained weights to a portable graph

export_onnx.py

```python
import torch, onnx

model = RideDurationTorchModel()
model.eval()          # disable dropout / batchnorm

dummy = torch.randn(1, 2)   # shape only

torch.onnx.export(
    model, dummy, "model.onnx",
    export_params=True,
    opset_version=17,
    input_names=["features"],
    output_names=["duration"],
    dynamic_axes={"features": {0: "batch"},
                  "duration": {0: "batch"}},
)

onnx.checker.check_model(onnx.load("model.onnx"))
```

model.eval()

Forgetting this leaves dropout in training mode — silently wrong predictions.

dummy_input

Only shape and dtype matter. PyTorch traces the graph by running it once.

dynamic_axes

Without it the model is locked to batch size 1.

onnx.checker

Validate before shipping. A broken export fails silently in some runtimes.

scikit-learn and inference

Export, then run anywhere

onnx_sklearn.py

```python
from skl2onnx import to_onnx
from skl2onnx.common.data_types import FloatTensorType

sk_model = RideDurationModel()._model   # fitted
init_type = [("features", FloatTensorType([None, 2]))]
onnx_model = to_onnx(sk_model, initial_types=init_type)

with open("model.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())

# Run it
import onnxruntime as ort, numpy as np

sess = ort.InferenceSession("model.onnx")
name_in  = sess.get_inputs()[0].name
name_out = sess.get_outputs()[0].name

batch = np.array([[5.0, 2]], dtype=np.float32)
out = sess.run([name_out], {name_in: batch})[0]
```

skl2onnx

Converts most sklearn estimators and Pipelines.

providers

Pick the backend: CPU, CUDA, TensorRT or OpenVINO — same file.

get_inputs()

Read names from the model. Never hardcode them.

Batch inference

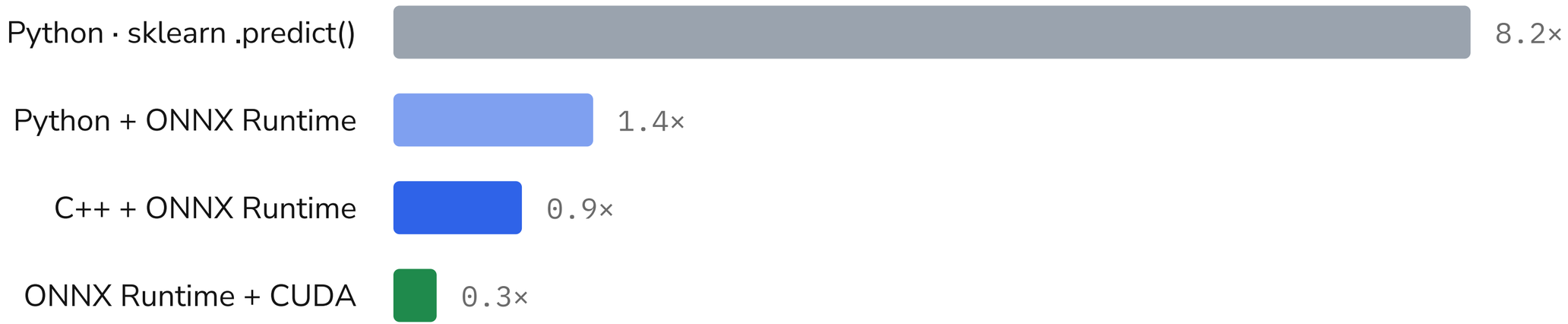
Only works because dynamic_axes was set at export.

What converts, and how well

SOURCE FRAMEWORK	CONVERTER	NOTES
PyTorch	`torch.onnx (built in)`	Best supported — native export API
scikit-learn	`skl2onnx`	Covers most estimators and Pipelines
XGBoost / LightGBM	`onnxmltools`	Tree ensembles convert well
TensorFlow / Keras	`tf2onnx`	Some custom layers need manual ops
Hugging Face	`optimum.onnxruntime`	Transformers export with attention support

What you actually gain

Same model, same inputs — only the runtime changes



Relative inference latency. The ratio is the point, not the absolute milliseconds.

When to use it, and when not to

USE ONNX WHEN

- You serve from C++, Java, C# or the browser
- You want to drop a training framework from your image
- You target edge devices — mobile, Raspberry Pi, embedded
- You need TensorRT, OpenVINO or CoreML acceleration
- Multiple teams need the model without your Python env

SKIP ONNX WHEN

- You are serving from Python anyway
- Your model uses custom ops the converter cannot handle
- You need to keep fine-tuning the same artifact
- The model changes weekly — one more CI step to maintain
- Your current latency is already well within SLA

Validation at the boundary

Never trust incoming data

src/schemas.py

```python
from pydantic import BaseModel, Field, field_validator
from typing import Annotated

class PredictRequest(BaseModel):
    distance_km: Annotated[float, Field(gt=0, le=500)]
    passengers:  Annotated[int,   Field(ge=1, le=8)]
    hour_of_day: Annotated[int,   Field(ge=0, le=23)]

    @field_validator("distance_km")
    @classmethod
    def not_zero(cls, v: float) -> float:
        if v < 0.1:
            raise ValueError("must be > 0.1 km")
        return round(v, 3)

# POST {"distance_km": -5} → 422
```

Field constraints

gt=0, le=500 — enforced before your function runs.

Custom validators

@field_validator for rules constraints cannot express.

Typed response

The response schema is your API contract.

Automatic 422

Invalid requests never reach your model.

Python trains. Production serves in C++.

Every serious deployment exports the model. Python’s role ends at the .onnx file.

Computer vision

C++ · CUDA

Tesla, Mobileye, Valeo, Bosch. TensorRT and ONNX Runtime C++. 60fps at 1080p leaves no room for Python overhead.

Embedded and edge

C · C++

ST, Qualcomm, ARM, NXP. A microcontroller has 256KB of RAM; the Python runtime alone needs 30MB.

LLM serving

C++ · CUDA · Rust

vLLM PagedAttention, llama.cpp, TensorRT-LLM, Mistral.rs. Python touches the request for ~2ms; CUDA runs for ~500ms.

Python = research and glue. C++/Rust/CUDA = production runtime. ONNX is the file that crosses the boundary.

TOPIC 04 · C++ ONNX RUNTIME

Real-time vision, in C++

How Valeo, Mobileye and Bosch run models in production vehicles

production_inference.cpp

```cpp
#include <onnxruntime_cxx_api.h>
#include <opencv2/opencv.hpp>

int main() {
    // 1. load the model once, at startup
    Ort::Env env(ORT_LOGGING_LEVEL_WARNING, "prod");
    Ort::SessionOptions opts;
    opts.SetIntraOpNumThreads(4);
    opts.AppendExecutionProvider_CUDA({});   // GPU path
    Ort::Session session(env, "model.onnx", opts);

    // 2. camera loop — must stay under 16ms
    cv::VideoCapture cap(0);
    cv::Mat frame, blob;
    while (cap.read(frame)) {
        cv::resize(frame, blob, {640, 640});
        auto out = session.Run(...);   // 2–8ms hot
    }
```

Model loaded once

Ort::Session is constructed at startup, never inside the loop. Cold start is 200–500ms; hot inference is 2–8ms per frame.

One line switches hardware

AppendExecutionProvider swaps CPU for CUDA, TensorRT, OpenVINO or DirectML. The inference code is unchanged.

Why not Python here

60fps at 1080p is 120MB/sec of frame data. The memcpy overhead alone would drop frames.

TOPIC 05

Docker and containerization

The number one MLOps skill

Ship the environment, not just the code

THE PROBLEM

“It works on my machine”

- Python 3.9 locally vs 3.11 on the server
- scikit-learn 1.2 vs 1.4 in production
- Missing system library (libgomp)
- Different OS path separators

THE DOCKER ANSWER

One image, every environment

- Frozen Python 3.11 and exact packages
- Same image dev → CI → staging → prod
- No install-guide documentation debt
- Runs anywhere Docker is installed

Image
Read-only template. Like a class.

Container
Running instance. Like an object.

Dockerfile
Recipe that builds the image, layer by layer.

Registry
Docker Hub or ECR — store and share images.

TOPIC 05 · INSTALL

Installing Docker

Get it running before we write the Dockerfile.

1 **Windows / macOS**

Download Docker Desktop

```
docs.docker.com/get-docker
```

Requires WSL 2 on Windows — the installer prompts you.

2 **Ubuntu / Debian**

Install via the official apt repo

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker
```

The usermod step lets you run docker without sudo.

3 **All platforms**

Verify the install

```
docker --version
docker run hello-world
```

You should see “Hello from Docker!” in your terminal.

4 **First commands**

Explore before we code

```
docker pull python:3.11-slim
docker images
docker ps
```

Pull an image, list images, list running containers.

Dockerfile anatomy

Every line is a frozen decision

Dockerfile

```
# Stage 1: build
FROM python:3.11-slim AS builder
WORKDIR /app
COPY pyproject.toml .
RUN pip install --no-cache-dir -e .[dev]

# Stage 2: runtime
FROM python:3.11-slim AS runtime
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.11 \
     /usr/local/lib/python3.11
COPY src/ ./src/

RUN adduser --disabled-password appuser
USER appuser

EXPOSE 8000
CMD ["uvicorn", "src.api:app", "--host", "0.0.0.0"]
```

Multi-stage build

Builder installs everything, runtime copies only what runs. 60% smaller.

Layer caching

COPY pyproject.toml before COPY src/. Deps change rarely, code changes often.

Non-root user

Never run as root. If the app is exploited, damage is contained.

CMD vs ENTRYPOINT

CMD is the default command. ENTRYPOINT is the fixed binary.

docker-compose

Your API and MLflow, one command

docker-compose.yml

```yaml
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      - MODEL_PATH=/models/v1/model.pkl
      - LOG_LEVEL=INFO
    volumes:
      - ./models:/models:ro
    depends_on: [mlflow]

  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.12.1
    ports: ["5000:5000"]
    volumes:
      - mlflow-data:/artifacts
```

Environment variables

Inject config at runtime. Never hardcode paths or secrets.

Read-only volumes

:ro means the API can read models but not modify them.

depends_on

API waits for MLflow. In production add healthcheck conditions.

Daily commands

docker compose up -d · ps · logs -f api

TOPIC 06

Logging and testing

print() is not logging, and untested code is not trusted

JSON logs are searchable. print() is not.

BAD — print()

```
print("Prediction made: 23.5")
print("ERROR: model not loaded")
```

GOOD — structlog

```
log.info("prediction.made", value=23.5,
         model_version="v1.2", latency_ms=4.1)
```

src/logging.py

```python
import structlog, logging

logging.basicConfig(
    format="%(message)s", level=logging.INFO,
)
structlog.configure(processors=[
    structlog.processors.add_log_level,
    structlog.processors.TimeStamper(fmt="iso"),
    structlog.processors.JSONRenderer(),
])

log = structlog.get_logger().bind(
    request_id=data.request_id,   # correlation ID
    endpoint="/predict",
)
log.info("predict.start",
distance=data.distance_km)
```

Unit testing with pytest

Code that cannot be tested cannot be trusted

tests/test_model.py

```python
import pytest
from unittest.mock import MagicMock
from src.model import RideDurationModel

def test_predict_returns_float():
    model = RideDurationModel()
    model._model = MagicMock()
    model._model.predict.return_value = [23.5]
    assert model.predict([5.0, 1]) == 23.5

@pytest.mark.parametrize("dist,pax,expected", [
    (1.0, 1, 5.2), (10.0, 2, 24.8), (0.5, 4, 3.1),
])
def test_predict_inputs(dist, pax, expected):
    model = RideDurationModel()
    model._model = MagicMock()
    model._model.predict.return_value = [expected]
    assert model.predict([dist, pax]) == expected
```

MagicMock

Never call real ML code in unit tests.

parametrize

One test function, many inputs.

Fixtures

Shared setup, injected by argument name.

assert semantics

pytest shows actual vs expected on failure.

Testing the API

Full request cycle, no running server

tests/test_api.py

```python
from fastapi.testclient import TestClient
from src.api import app

client = TestClient(app)

def test_predict_success():
    resp = client.post("/predict",
        json={"distance_km": 5.0, "passengers": 2})
    assert resp.status_code == 200
    assert "duration_min" in resp.json()

def test_predict_negative_distance():
    resp = client.post("/predict",
        json={"distance_km": -1.0, "passengers": 1})
    assert resp.status_code == 422   # Pydantic catches it

# pytest --cov=src --cov-report=html
```

TestClient

No server needed — requests are processed in-process.

Test the contract

Happy path and the rejection path, both.

Status codes

200 for success, 422 for a schema violation.

What good coverage looks like

Coverage measures test thoroughness — not correctness.

≥ 80%

Minimum threshold for a CI gate

≥ 90%

Recommended for core business logic

< 70%

Red flag — what is not being tested?

100%

Achievable for schemas and pure functions

Coverage target for the course project: at least 80% across src/

SESSION PROJECT

The containerized ML service

Take a trained model and make it production-ready

SESSION 1 · PROJECT

What you build today

01 **Python package**

Clean src/ layout with pyproject.toml. pip install -e . works.

02 **FastAPI service**

/predict, /health, /feedback endpoints with Pydantic schemas.

03 **Docker**

Multi-stage Dockerfile plus docker-compose.yml. docker compose up works.

04 **pytest suite**

Unit and API tests. At least 80% coverage. pytest --cov runs clean.

05 **Structured logging**

JSON logs with request_id, endpoint and latency_ms on every request.

06 **README**

Setup in three commands. Architecture diagram. What the model predicts.

REFERENCE · DOCKER

Docker quick reference

Commands you will use every day

`docker build -t my-api .`	Build image from the Dockerfile here
`docker run -d --name api my-api`	Run detached, with a name
`docker ps -a`	List all containers, including stopped
`docker exec -it api /bin/bash`	Shell into a running container
`docker images`	List local images
`docker compose up -d`	Start every service
`docker run -p 8000:8000 my-api`	Run, mapping host:8000 → container:8000
`docker ps`	List running containers
`docker logs -f api`	Tail live logs
`docker stop api && docker rm api`	Stop and remove a container
`docker rmi my-api`	Delete an image
`docker compose down -v`	Stop services and remove volumes

REFERENCE · PYTEST

pytest quick reference

Run, filter, measure

`pytest`	Run all tests here
`pytest tests/test_model.py`	Run one file
`pytest -k "predict"`	Run tests whose name contains predict
`pytest --cov=src`	Run with coverage for src/
`pytest --cov=src --cov-fail-under=80`	Fail CI below 80%
`pytest --tb=short`	Short traceback

`pytest -v`	Verbose — show each test name
`pytest tests/test_model.py::test_predict`	Run one test
`pytest -m slow`	Run tests marked @pytest.mark.slow
`pytest --cov=src --cov-report=html`	HTML report in htmlcov/
`pytest -x`	Stop at the first failure
`pytest -s`	Show print() output

RESOURCES

Where to go deeper

FastAPI
fastapi.tiangolo.com/tutorial
The best framework docs I have read

pytest
docs.pytest.org
Fixtures, parametrize, marks

Pydantic v2
docs.pydantic.dev
Field validators, model config, JSON schema

MLOps Zoomcamp
github.com/DataTalksClub/mlops-zoomcamp
The free 9-week course this aligns with

Docker
docs.docker.com/get-started
Official Getting Started series

structlog
structlog.org
The structured logging library used here

Made With ML
madewithml.com
End-to-end MLOps reference — free

TechWorld with Nana
youtube.com/@TechWorldwithNana
3-hour Docker crash course

NEXT · SESSION 2

You built the service. Session 2 automates it.

The Docker image you built today is exactly what Session 2's pipeline will build, test and push — automatically, on every git push.

MLflow

Track every experiment. Register the best model.

DVC

Version data alongside code. Reproduce from a hash.

GitHub Actions

Lint → test → build → push on every PR.

Terraform

Provision ML infrastructure with code.

CT pipeline

Auto-retrain when data drifts.

Before you leave: push your repo, confirm docker compose up runs, make sure pytest passes.

Questions?

Session 1 complete. Now go build.

Aya · MLOps MENA Community

https://www.linkedin.com/company/mlops-mena #mlops-mena